



Università dell' Aquila

ROOC: linguaggio ed ambiente integrato per la
modellazione e soluzione di problemi di ottimiz-
zazione lineare

Department of Information Engineering, Computer Science and Mathematics
Corso di Laurea in Computer Science

Candidato

Enrico Menichelli

Matricola 279061

Relatore

Prof. Matteo Spezialetti

Correlatore

Prof. Stefano Smriglio

Anno Accademico 2025/2026

**ROOC: linguaggio ed ambiente integrato per la modellazione e soluzione di
problemi di ottimizzazione lineare**

Tesi di Laurea. Università dell' Aquila

© 2025 Enrico Menichelli. Tutti i diritti riservati

Questa tesi è stata composta con L^AT_EX e la classe uaqthesis.

Email dell'autore: specy.dev@gmail.com

Contents

1	Introduzione	1
1.1	Contesto e motivazione	1
1.2	Il problema e l'idea di ROOC	1
1.3	Obiettivi e contributi	2
1.4	Struttura della tesi	2
2	Background e stato dell'arte	3
2.1	Programmazione lineare e MILP	3
2.2	I modelli di ottimizzazione	5
2.3	Linguaggi e strumenti di modellazione esistenti	5
2.4	Cenni di teoria dei linguaggi e dei compilatori	7
2.5	Posizionamento di ROOC	9
3	Il linguaggio ROOC	10
3.1	Principi di design	10
3.2	Struttura di un modello	11
3.3	Esempio guida	12
3.4	Primitive e tipi di dato	13
3.5	Iteratori, <i>block functions</i> e destructuring	14
3.6	Operatori e definizione delle variabili	16
3.7	Libreria standard e funzioni definite dall'utente	17
4	Architettura del compilatore	18
4.1	La pipeline multi-fase	18

4.2	L'astrazione <i>Pipe</i>	19
4.3	Estensibilità e visualizzazione passo-passo	20
5	Analisi sintattica e rappresentazione intermedia	22
5.1	Concetti teorici	22
5.2	Applicazione in ROOC: la grammatica PEST	23
5.3	Il PreModel e l'Intermediate Language	25
5.4	Gestione e tracciamento degli errori	26
6	Analisi semantica: tipi e trasformazione del modello	27
6.1	Concetti teorici	27
6.2	Applicazione in ROOC: il sistema dei tipi	27
6.3	Il type checker	28
6.4	Trasformazione PreModel $\rightarrow$ Model	28
7	Linearizzazione e forma standard	29
7.1	Concetti teorici	29
7.2	Applicazione in ROOC: da Model a LinearModel	29
7.3	Linearizzazione di min, max e valore assoluto	30
7.4	Trasformazione in forma standard	30
8	Risoluzione dei modelli	31
8.1	Concetti teorici	31
8.2	Applicazione in ROOC: il semplice a due fasi	31
8.3	microlp: fork ed estensione a MILP	32
8.4	Backend esterni e <i>auto-solver</i>	32
8.5	Confronto qualitativo	32
9	Dalla libreria alla piattaforma web	33
9.1	Compilazione in WebAssembly e la libreria TypeScript	33
9.2	L'IDE web e l'LSP nel browser	33
9.3	Valore didattico	34

10 Conclusioni e sviluppi futuri	35
10.1 Riepilogo dei contributi	35
10.2 Limitazioni	35
10.3 Sviluppi futuri	35
Appendices	36
A Grammatica completa di ROOC	36
B Esempi di modelli	37
Bibliography	38

Chapter 1

Introduzione

Capitolo introduttivo: presentare il contesto, il problema, gli obiettivi e i contributi della tesi. Stile discorsivo, una pagina o due.

1.1 Contesto e motivazione

I problemi di ottimizzazione sono pervasivi in ambito industriale, logistico e scientifico, ma la loro formalizzazione in un modello matematico risolubile richiede competenze specialistiche e strumenti spesso poco accessibili. In questo capitolo si introduce il contesto della *Ricerca Operativa* e dell'*Ottimizzazione Combinatoria* da cui nasce il progetto.

Descrivere brevemente perché modellare problemi di ottimizzazione è difficile e perché serve un linguaggio dedicato.

1.2 Il problema e l'idea di ROOC

Introdurre ROOC: un linguaggio di modellazione formale che, a partire da un modello e dai relativi dati, produce un modello lineare risolubile con tecniche di ottimizzazione. Anticipare l'idea della pipeline di compilazione.

1.3 Obiettivi e contributi

Elencare in modo puntuale i contributi: (1) il design del linguaggio, (2) il compilatore multi-fase, (3) il solver MILP `microlp`, (4) la libreria riusabile (Rust/WASM/TypeScript) e la piattaforma web didattica.

1.4 Struttura della tesi

Riassumere il contenuto di ciascun capitolo successivo.

Chapter 2

Background e stato dell'arte

Questo capitolo fornisce le basi necessarie a comprendere il resto della tesi. Si introducono dapprima i concetti fondamentali della programmazione lineare e della sua estensione intera (Sezioni 2.1 e 2.2); si analizzano poi i principali linguaggi e strumenti di modellazione esistenti (Sezione 2.3), per inquadrare il contesto in cui si colloca ROOC; infine si richiamano i concetti di teoria dei compilatori (Sezione 2.4) che verranno applicati nei capitoli successivi, e si delinea il posizionamento del progetto (Sezione 2.5).

2.1 Programmazione lineare e MILP

La *programmazione lineare* (*Linear Programming*, LP) è il ramo dell'ottimizzazione matematica che studia la minimizzazione (o massimizzazione) di una funzione obiettivo lineare soggetta a un insieme di vincoli lineari. Un problema di programmazione lineare può essere espresso nella forma generale

$$\begin{aligned} \min_x \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b, \\ & x \geq 0, \end{aligned} \tag{2.1}$$

dove $x \in \mathbb{R}^n$ è il vettore delle *variabili decisionali*, $c \in \mathbb{R}^n$ il vettore dei coefficienti della funzione obiettivo, $A \in \mathbb{R}^{m \times n}$ la matrice dei coefficienti dei vincoli e $b \in \mathbb{R}^m$ il vettore dei termini noti. L'insieme dei punti che soddisfano tutti i vincoli prende il

nome di *regione ammissibile*; quando non è vuota e la funzione obiettivo è limitata sulla regione, esiste almeno una *soluzione ottima*, che il problema si propone di determinare [3, 2].

Un risultato classico della teoria afferma che, se esiste una soluzione ottima, allora ne esiste una in corrispondenza di un *vertice* del poliedro che descrive la regione ammissibile. Su questa proprietà si fonda il *metodo del semplice*, che esplora i vertici della regione spostandosi di volta in volta verso quello con valore migliore della funzione obiettivo. Il semplice opera su una rappresentazione del problema detta *forma standard*,

$$\min_x c^\top x \quad \text{s.t.} \quad Ax = b, \quad x \geq 0, \quad (2.2)$$

in cui tutti i vincoli sono uguaglianze e tutte le variabili sono non negative. La conversione di un problema dalla forma (2.1) alla forma (2.2) è un passaggio puramente meccanico, che verrà trattato nel Capitolo 7 come una delle fasi della pipeline di compilazione di ROOC.

In molte applicazioni reali le variabili decisionali non possono assumere valori frazionari: rappresentano scelte indivisibili (quanti veicoli acquistare) o decisioni di tipo sì/no (se attivare o meno un impianto). Si parla allora di *programmazione lineare intera mista* (*Mixed-Integer Linear Programming*, MILP), in cui si aggiunge il vincolo di integralità su un sottoinsieme $I \subseteq \{1, \dots, n\}$ delle variabili:

$$\min_x c^\top x \quad \text{s.t.} \quad Ax \leq b, \quad x \geq 0, \quad x_j \in \mathbb{Z} \quad \forall j \in I. \quad (2.3)$$

Un caso particolarmente importante è quello delle variabili *binarie*, in cui $x_j \in \{0, 1\}$, usate per modellare decisioni logiche. L'aggiunta del vincolo di integralità cambia radicalmente la natura del problema: mentre la programmazione lineare continua è risolvibile in tempo polinomiale, la programmazione lineare intera è in generale *NP-hard* [15, 11]. La tecnica più diffusa per risolverla è il *branch-and-bound*, che esplora in modo sistematico lo spazio delle soluzioni intere risolvendo una successione di rilassamenti continui; tale tecnica sarà ripresa nel Capitolo 8, dove si descrive l'estensione del solver *microlp* al caso intero e binario.

2.2 I modelli di ottimizzazione

Tradurre un problema del mondo reale in una delle forme (2.1) o (2.3) significa costruirne un *modello*: identificare le variabili decisionali, esprimere l'obiettivo come funzione lineare di tali variabili e codificare i requisiti del problema come vincoli. Questa attività di *modellazione* è concettualmente distinta dalla *risoluzione* numerica, ed è proprio il livello a cui opera un linguaggio come ROOC.

Si consideri, a titolo di esempio, il classico problema dello *zaino* (*knapsack*): dato un insieme di n oggetti, ciascuno con un valore v_i e un peso w_i , e uno zaino di capacità C , si vuole selezionare il sottoinsieme di oggetti di valore complessivo massimo il cui peso non superi la capacità. Introducendo una variabile binaria $x_i \in \{0, 1\}$ che indica se l'oggetto i -esimo è selezionato, il modello si scrive

$$\begin{aligned} \max \quad & \sum_{i=1}^n v_i x_i \\ \text{s.t.} \quad & \sum_{i=1}^n w_i x_i \leq C, \\ & x_i \in \{0, 1\} \quad \forall i \in \{1, \dots, n\}. \end{aligned} \tag{2.4}$$

Si noti come la struttura del modello sia indipendente dai dati specifici (n , i valori v_i , i pesi w_i , la capacità C): la stessa formulazione descrive un'intera famiglia di istanze. La separazione tra la *struttura* del modello e i *dati* di una particolare istanza è un principio cardine dei linguaggi di modellazione, ROOC compreso, e ne motiva i costrutti come gli iteratori e le sommatorie indicizzate, descritti nel Capitolo 3. Il problema dello zaino, insieme al problema del *dominating set* su grafo, farà da esempio guida nel resto della tesi.

2.3 Linguaggi e strumenti di modellazione esistenti

La modellazione di problemi di ottimizzazione è supportata da un'ampia gamma di strumenti, che si possono collocare lungo uno spettro che va dalle rappresentazioni di basso livello, vicine al solver, fino ai linguaggi algebrici di alto livello, vicini alla notazione matematica.

Formati di scambio di basso livello. Formati testuali come il *CPLEX LP format* e il formato *MPS* descrivono direttamente la matrice dei coefficienti di un modello lineare. Sono pensati come interfaccia tra strumenti software e solver, non per essere scritti o letti dall'uomo: non offrono astrazioni quali iteratori, costanti simboliche o strutture dati, e impongono di esplicitare ogni singolo vincolo. ROOC è in grado di esportare in formato CPLEX LP, usato come interfaccia verso solver esterni.

Linguaggi algebrici di modellazione. I *linguaggi algebrici di modellazione* (*Algebraic Modeling Languages*, AML) permettono di esprimere i modelli in una forma vicina a quella matematica, con sommatorie, indici e parametri simbolici. **AMPL** è il capostipite di questa famiglia: sviluppato presso i Bell Labs, separa nettamente modello e dati e si interfaccia con un gran numero di solver [6]. Si tratta però di un prodotto commerciale e a sorgente chiuso. **GNU MathProg** (GMPL) ne riprende un sottoinsieme in forma libera, come parte del GNU Linear Programming Kit [10]. **GAMS** è un altro storico ambiente commerciale della stessa categoria.

Linguaggi embedded. Una tendenza più recente è quella di offrire la modellazione come libreria all'interno di un linguaggio di programmazione general-purpose. **Pyomo** fornisce un'API a oggetti in Python [8], mentre **JuMP**, embedded in Julia, sfrutta la metaprogrammazione del linguaggio ospite per ottenere al contempo espressività e prestazioni elevate [4]. Questo approccio eredita gratuitamente l'ecosistema del linguaggio ospite, ma lega la sintassi del modello a quella di tale linguaggio.

Linguaggi di modellazione a vincoli. **MiniZinc** è un linguaggio open source per la *programmazione a vincoli*, indipendente dal solver: un modello MiniZinc viene compilato in un formato intermedio di più basso livello (FlatZinc) che diversi solver, di tipo CP, MIP o SAT, possono poi risolvere [12]. Questo schema, in cui un linguaggio di alto livello viene *compilato* verso una forma intermedia solver-indipendente, è concettualmente vicino all'architettura di ROOC.

Solver e relative API. Va infine distinto il livello del *solver* vero e proprio. Strumenti come **Gurobi** [7] o **HiGHS** [9] sono motori di risoluzione, corredati di API per la costruzione dei modelli, ma non costituiscono di per sé linguaggi di modellazione. ROOC si appoggia a solver di questo tipo come back-end intercambiabili (Capitolo 8).

Rispetto a questo panorama, ROOC si propone di unire alcune caratteristiche che raramente coesistono in un unico strumento: una sintassi dichiarativa vicina alla matematica, ma con un sistema di tipi ricco che include strutture dati come *grafi*, tuple e iterabili; l'indipendenza dal solver tramite una pipeline di compilazione esplicita; l'esecuzione interamente lato client tramite WebAssembly; e un taglio didattico, con la possibilità di ispezionare ogni fase intermedia della risoluzione. Tali aspetti sono ripresi nella Sezione 2.5.

2.4 Cenni di teoria dei linguaggi e dei compilatori

ROOC è, a tutti gli effetti, un *compilatore*: traduce il linguaggio di modellazione, di alto livello, in rappresentazioni progressivamente più vicine a una forma risolvibile da un solver. La pipeline non è vincolata a un'unica forma di destinazione: l'implementazione attuale arriva fino a un modello lineare in forma standard, ma la stessa architettura potrebbe in futuro produrre modelli non lineari, fermarsi a una fase intermedia o aggiungere ulteriori trasformazioni. È quindi utile richiamare le fasi classiche di un compilatore [1], perché esse forniscono la struttura portante della pipeline descritta nel resto della tesi.

Analisi lessicale e sintattica. Le prime fasi di un compilatore riconoscono la struttura del testo sorgente. L'*analisi lessicale* suddivide il flusso di caratteri in unità elementari (*token*); l'*analisi sintattica* (*parsing*) verifica che la sequenza di token rispetti la grammatica del linguaggio e ne costruisce una rappresentazione ad albero, l'*albero di sintassi astratta* (*Abstract Syntax Tree*, AST). La grammatica è tradizionalmente espressa tramite una grammatica *context-free* (CFG), spesso in notazione BNF.

Parsing Expression Grammar. Una formalizzazione alternativa è data dalle *Parsing Expression Grammar* (PEG) [5]. A differenza delle CFG, le PEG sono intrinsecamente non ambigue: la scelta tra alternative è *ordinata*, ossia si prova la prima alternativa e si passa alla successiva solo in caso di fallimento. Inoltre le PEG integrano analisi lessicale e sintattica in un'unica grammatica, eliminando la necessità di un lexer separato. Queste proprietà le rendono particolarmente adatte a un linguaggio come ROOC; la grammatica concreta, espressa con la libreria PEST, è descritta nel Capitolo 5.

Analisi semantica e sistema dei tipi. Superata l'analisi sintattica, l'*analisi semantica* verifica che il programma sia semanticamente corretto, cioè che le regole del linguaggio vengano rispettate: che le variabili siano dichiarate, che le operazioni siano applicate a operandi di tipo compatibile, che le funzioni siano invocate con il numero e il tipo di argomenti corretti. Il *sistema dei tipi* è lo strumento formale con cui si esprimono e si verificano queste proprietà [13]. In ROOC questa fase comprende anche la *trasformazione* del modello, in cui costanti, iteratori e sommatorie vengono risolti ed espansi (Capitolo 6).

Rappresentazione intermedia e lowering. I compilatori non traducono direttamente dall'AST al codice finale, ma passano attraverso una o più *rappresentazioni intermedie* (IR), via via più vicine alla macchina di destinazione. Il processo di traduzione da una rappresentazione di livello superiore a una di livello inferiore prende il nome di *lowering*. Nel caso di ROOC, il *lowering* consiste nella *linearizzazione* del modello, cioè l'eliminazione dei costrutti non lineari come `min`, `max` e valore assoluto, e nella sua riduzione alla forma standard, descritte nel Capitolo 7.

Generazione del codice. L'ultima fase di un compilatore tradizionale produce il codice oggetto. In ROOC il ruolo del codice oggetto è svolto dal modello lineare in forma standard, che viene infine dato in pasto a un solver per ottenere la soluzione, come discusso nel Capitolo 8.

2.5 Posizionamento di ROOC

Alla luce di quanto esposto, ROOC si colloca come linguaggio di modellazione di alto livello, indipendente dal solver, con alcune scelte progettuali distintive:

- una notazione dichiarativa vicina alla matematica, ma arricchita da un sistema di tipi che comprende strutture dati di alto livello (come grafi, tuple e iterabili) utili a modellare problemi di ottimizzazione combinatoria;
- un'architettura a compilatore con pipeline esplicita e *solver-indipendente*, sulla scia dell'approccio di MiniZinc, ma orientata alla programmazione lineare intera;
- l'esecuzione interamente lato client tramite la compilazione in WebAssembly [14], che elimina la necessità di un'infrastruttura server e rende lo strumento immediatamente accessibile da un browser;
- un taglio *didattico*: la possibilità di ispezionare ogni rappresentazione intermedia e di visualizzare passo dopo passo l'esecuzione del simplesso rende ROOC uno strumento utile non solo per risolvere modelli, ma anche per comprenderne il processo di risoluzione.

I capitoli successivi approfondiscono ciascuno di questi aspetti, a partire dalla descrizione del linguaggio dal punto di vista dell'utente (Capitolo 3).

Chapter 3

Il linguaggio ROOC

Questo capitolo descrive il linguaggio ROOC dal punto di vista di chi lo utilizza: quali costrutti mette a disposizione e quale ne è il significato. La realizzazione interna di tali costrutti, ossia il modo in cui vengono analizzati e tradotti dal compilatore, è rimandata ai capitoli successivi. Per tutti gli esempi si fa riferimento alla sintassi effettivamente accettata dalla grammatica del linguaggio (Capitolo 5).

3.1 Principi di design

Il progetto del linguaggio è guidato da alcuni principi di fondo. Il primo è la *vicinanza alla notazione matematica*: un modello scritto in ROOC deve ricordare il più possibile la formulazione che se ne darebbe su carta, con funzione obiettivo, vincoli e sommatorie indicizzate. Il secondo è la *separazione tra struttura del modello e dati*: la formulazione di un problema è indipendente dai valori numerici della singola istanza, che vengono forniti separatamente. Il terzo è la presenza di un *sistema di tipi statico*, che consente di individuare errori (variabili non dichiarate, operazioni tra tipi incompatibili, funzioni invocate in modo scorretto) prima ancora di tentare la risoluzione. Infine, il linguaggio è pensato per essere *estensibile*: oltre alla libreria di funzioni predefinite, l'utente può definire le proprie funzioni e le proprie costanti.

3.2 Struttura di un modello

Un modello ROOC è composto da una parte obbligatoria, che definisce l'obiettivo e i vincoli, e da due blocchi opzionali, `where` e `define`, che forniscono rispettivamente i dati e la dichiarazione del dominio delle variabili. La struttura generale è la seguente:

```
min/max <espressione obiettivo>
s.t.
    <vincoli>
where
    <dichiarazione delle costanti>
define
    <dichiarazione dei domini delle variabili>
```

Listing 3.1. Struttura generale di un modello ROOC.

Obiettivo. La prima riga dichiara l'obiettivo, introdotto dalla parola chiave `min` o `max` seguita dall'espressione da ottimizzare. In alternativa è ammessa la parola chiave `solve`, che richiede di trovare una qualunque soluzione ammissibile senza ottimizzare alcuna funzione obiettivo.

Vincoli. La sezione introdotta da `s.t.` (oppure `subject to`) contiene l'elenco dei vincoli. Ogni vincolo confronta due espressioni tramite un operatore di relazione (`<=`, `>=`, `=`, `<`, `>`). Un vincolo può essere preceduto da un nome (nella forma `nome: . . .`) e può essere seguito da una clausola di iterazione `for`, che lo replica per ogni valore degli indici, generando così un'intera famiglia di vincoli a partire da un'unica riga.

Blocco where. Il blocco opzionale `where` dichiara le *costanti* del modello, ciascuna con la sintassi `let nome = valore`. Il valore può essere una qualunque espressione del linguaggio: oltre a primitive come numeri, stringhe, array (anche multidimensionali) e grafi, sono ammesse espressioni più articolate, come chiamate di funzione o operazioni tra altre costanti. È qui che vengono tipicamente forniti i dati della specifica istanza, mantenendoli separati dalla struttura del modello.

Blocco define. Il blocco opzionale `define` dichiara il *dominio* delle variabili decisionali tramite la parola chiave `as`, come descritto nella Sezione 3.6. Anche le dichiarazioni di dominio possono essere indicizzate con una clausola `for`.

3.3 Esempio guida

Per illustrare in modo concreto i costrutti del linguaggio si introducono due esempi che verranno richiamati nel resto della tesi: il problema dello zaino e il problema del *dominating set* su grafo.

Il Listato 3.2 riprende il modello dello zaino già formalizzato nella Sezione 2.2. I dati dell'istanza (pesi, valori e capacità) sono dichiarati nel blocco `where`, mentre la variabile binaria x_i è dichiarata nel blocco `define`, una per ciascun oggetto.

```
max sum((value, i) in enumerate(values)) { value * x_i }
s.t.
    sum((weight, i) in enumerate(weights)) { weight * x_i } ≤
        capacity
where
    let weights = [10, 60, 30, 40, 30, 20, 20, 2]
    let values = [1, 10, 15, 40, 60, 90, 100, 15]
    let capacity = 102
define
    x_i as Boolean for i in 0..len(weights)
```

Listing 3.2. Il problema dello zaino in ROOC.

Il secondo esempio (Listato 3.3) mostra l'espressività del linguaggio sui grafi. Dato un grafo G , il problema del *dominating set* chiede di selezionare il minimo insieme di nodi tale che ogni nodo sia selezionato oppure adiacente a un nodo selezionato. Il grafo è dichiarato come costante, e i vincoli vengono generati per ogni nodo tramite la clausola `for`.

```
min sum(u in nodes(G)) { x_u }
s.t.
    x_v + sum((_, u) in neigh_edges(v)) { x_u } ≥ 1 for v in
        nodes(G)
```

```
where
  let G = Graph {
    A → [B, C, D, E, F],
    B → [A, E, C, D, J],
    C → [A, B, D, E, I]
  }
define
  x_u, x_v as Boolean for v in nodes(G), (_, u) in neigh_edges
    (v)
```

Listing 3.3. Il problema del dominating set in ROOC.

3.4 Primitive e tipi di dato

Il linguaggio dispone di un insieme di tipi di dato primitivi, usati per le costanti, i risultati delle funzioni e i valori manipolati durante la trasformazione del modello. I tipi disponibili sono:

- i tipi numerici `Number` (numero in virgola mobile), `Integer` (intero con segno) e `PositiveInteger` (intero non negativo);
- `String`, le stringhe di caratteri;
- `Boolean`, i valori di verità `true` e `false`;
- `Graph`, `GraphNode` e `GraphEdge`, che rappresentano rispettivamente un grafo, un suo nodo e un suo arco;
- `Tuple`, sequenze di lunghezza fissa che possono contenere valori di tipi diversi;
- `Iterable`, le collezioni omogenee (per esempio gli array e gli intervalli) su cui è possibile iterare.

I tipi numerici sono distinti perché ciò consente al sistema dei tipi di essere più preciso: per esempio, la funzione `len` restituisce un `PositiveInteger`, garantendo staticamente la non negatività del risultato. La conversione tra tipi numerici avviene in modo automatico quando necessario.

Array. Gli array si scrivono tra parentesi quadre, come in `[10, 60, 30]`, e possono essere annidati per rappresentare matrici, come in `[[1, 2], [3, 4]]`. A livello di tipo, un array è un `Iterable` i cui elementi hanno tutti lo stesso tipo.

Grafi. Un grafo si dichiara con la parola chiave `Graph` seguita dall'elenco dei nodi e dei rispettivi archi uscenti. Ogni nodo ha la forma `Nodo -> [vicino1, vicino2, . . . ]`; a ciascun arco può essere associato un costo, indicato dopo il carattere `:`, come nel Listato 3.4.

```
let G = Graph {  
  A -> [B, C:2.5],  
  B -> [A],  
  C -> []  
}
```

Listing 3.4. Dichiarazione di un grafo con costi sugli archi.

3.5 Iteratori, *block functions* e destructuring

Iteratori. Gli iteratori sono il meccanismo con cui il linguaggio esprime in modo compatto famiglie di vincoli, di variabili o di termini di una sommatoria. La forma generale è `for tupla in iterabile`; l'iterabile può essere un intervallo numerico, scritto con `..` (estremo superiore escluso) o `..=` (estremo superiore incluso), oppure una qualunque collezione, tipicamente prodotta da una funzione come `nodes(G)` o `enumerate(. . .)`. È possibile concatenare più iteratori separandoli con la virgola.

```
for i in 0..10           // interi da 0 a 9 (estremo superiore  
  escluso)  
for i in 0..=10          // interi da 0 a 10 (estremo superiore  
  incluso)  
for v in nodes(G)        // i nodi di un grafo  
for (value, i) in enumerate(values) // coppie (elemento,  
  indice)  
for i in 0..n, j in 0..n // due iteratori concatenati
```

Listing 3.5. Esempi di iteratori.

Block functions e nozione di *scope*. Le *block function* sono funzioni di aggregazione il cui risultato è una singola espressione. Il linguaggio ne offre due forme, che si distinguono per la presenza o l'assenza di uno *scope*, cioè di un insieme di variabili di iterazione introdotte localmente.

Le *block scoped function* possiedono uno scope: hanno la forma `nome(iteratori){corpo}`, dove gli iteratori dichiarano una o più variabili visibili soltanto all'interno del corpo. Il corpo viene valutato una volta per ogni combinazione di valori degli iteratori, e i risultati sono combinati dall'operatore di aggregazione. Sono disponibili nelle varianti `sum`, `prod`, `min`, `max` e `avg`.

```
sum((value, i) in enumerate(values)) { value * x_i }
```

Listing 3.6. Block scoped function: lo scope introduce la variabile `i` e l'elemento `value`.

Le *block function* prive di scope, invece, operano su un elenco di espressioni già note, senza introdurre nuove variabili: hanno la forma `nome{a, b, c}` e sono disponibili per `min`, `max` e `avg`.

```
max{x_1, x_2, x_3}
```

Listing 3.7. Block function senza scope: opera su espressioni elencate esplicitamente.

La differenza è dunque sostanziale: nella forma con scope il numero di termini aggregati dipende dalla cardinalità degli iteratori e non è noto staticamente, mentre nella forma senza scope i termini sono quelli esplicitamente elencati. Per questo motivo le aggregazioni `sum` e `prod`, che hanno senso soprattutto su insiemi di cardinalità arbitraria, sono disponibili soltanto nella forma con scope.

Variabili composte. La notazione x_i , detta *variabile composta*, si scrive `x_i` e indica una variabile indicizzata. Gli indici possono essere identificatori o numeri (`x_i`, `x_0`) oppure espressioni racchiuse tra parentesi graffe (`x_{i+1}`). Durante la trasformazione del modello ogni combinazione concreta degli indici dà luogo a una variabile distinta del modello: iterando `x_i` su `i in 0..3` si ottengono le variabili `x_0`, `x_1` e `x_2`. Gli indici vengono valutati nel contesto di iterazione e concatenati a formare il nome finale della variabile; questo meccanismo è descritto in dettaglio

nel Capitolo 6. La forma *escaped* `\x_y` permette di trattare letteralmente un nome contenente l'underscore, disattivando l'espansione automatica.

Destructuring. Quando un iteratore produce tuple, è possibile decomporne le componenti direttamente nella dichiarazione, come in `(value, i) in enumerate(values)`: a ogni iterazione `value` assume il valore dell'elemento e `i` il suo indice. Il simbolo `_` indica una componente da ignorare, come in `(_, u) in neigh_edges(v)`, dove interessa solo il nodo di arrivo dell'arco.

3.6 Operatori e definizione delle variabili

Operatori. Sul piano delle espressioni il linguaggio mette a disposizione gli operatori aritmetici `+`, `-`, `*` e `/` e l'operatore unario di negazione. È supportata anche la *moltiplicazione implicita*: una scrittura come `2x_i` è interpretata come `2 * x_i`. Gli operatori sono *sovraccaricati* sui diversi tipi: oltre al consueto significato aritmetico sui tipi numerici, l'operatore `+` applicato a due stringhe ne produce la concatenazione. Le operazioni insiemistiche tra collezioni non sono espresse tramite operatori, ma tramite le funzioni `union`, `intersection` e `difference` (Sezione 3.7).

Tra gli sviluppi futuri del linguaggio si prevede l'introduzione di operatori *logici*, che permetterebbero di modellare in modo più naturale i problemi di ottimizzazione combinatoria in cui i vincoli hanno natura logica.

Dichiarazione delle variabili. Nel blocco `define` ogni variabile decisionale viene associata a un dominio tramite la parola chiave `as`. I domini disponibili sono:

- `Boolean`: variabile binaria, che assume valore 0 o 1;
- `Real(min, max)`: variabile reale compresa tra i due estremi indicati; se gli estremi sono omessi (`Real`) la variabile è illimitata;
- `NonNegativeReal(min, max)`: variabile reale non negativa; se gli estremi sono omessi assume valori nell'intervallo $[0, +\infty)$;
- `IntegerRange(min, max)`: variabile intera compresa tra i due estremi, che in questo caso sono obbligatori.

Gli estremi possono essere espressioni, anche dipendenti da costanti o da indici di iterazione. Una dichiarazione come `x_i as NonNegativeReal(Fmin[i], Fmax[i])` `for i in 0..n` associa così a ciascuna variabile `x_i` i propri estremi, letti dagli array `Fmin` e `Fmax`.

3.7 Libreria standard e funzioni definite dall'utente

Libreria standard. Il linguaggio offre una libreria di funzioni predefinite, utilizzabili nelle espressioni del modello. Le principali si possono raggruppare in tre famiglie. Le funzioni su *collezioni* comprendono `len` (numero di elementi), `enumerate` (alias `enum`, che associa a ogni elemento il suo indice), `zip` (che combina più collezioni elemento per elemento), `range` (che genera un intervallo numerico) e le operazioni insiemistiche `union`, `intersection` e `difference`. Le funzioni sui *grafi* comprendono `nodes` (alias `V`), `edges` (alias `E`), `neigh_edges` (alias `N`, che restituisce gli archi incidenti a un nodo) e `neigh_edges_of` (alias `N_of`). La disponibilità di alias come `V`, `E` e `N` avvicina la notazione a quella usuale della teoria dei grafi.

Funzioni definite dall'utente. Oltre alla libreria standard, l'utente può estendere il linguaggio con proprie funzioni e costanti. Questa possibilità è offerta soprattutto in ambiente web (Capitolo 9), dove le funzioni si dichiarano in TypeScript tramite l'API `makeRoocFunction`, specificandone il nome, i tipi dei parametri, il tipo restituito e l'implementazione. Le funzioni così registrate diventano disponibili nelle espressioni del modello allo stesso modo di quelle predefinite, e partecipano alla verifica statica dei tipi. Il meccanismo che rende possibile questa integrazione, basato sulla compilazione in WebAssembly e su un'interfaccia comune tra Rust e JavaScript, è descritto nel Capitolo 9.

Chapter 4

Architettura del compilatore

Prima di affrontare nel dettaglio le singole fasi di compilazione, questo capitolo ne offre la visione d'insieme. Si descrive come la traduzione di un modello sia organizzata in una sequenza di trasformazioni e come tale sequenza sia realizzata, a livello implementativo, da un'astrazione componibile che chiameremo *pipe*. I capitoli da 5 a 8 approfondiscono poi ciascuna fase.

4.1 La pipeline multi-fase

La compilazione di un modello ROOC non avviene in un unico passo, ma attraverso una successione di *rappresentazioni intermedie*, ognuna delle quali raffina la precedente avvicinandola a una forma risolvibile. Ogni rappresentazione è un tipo di dato a sé stante: il codice sorgente (`String`) viene dapprima associato a un parser, da cui si ottiene il `PreModel`, frutto della sola analisi sintattica; il `PreModel` viene poi sottoposto ad analisi semantica e trasformato nel `Model`; quest'ultimo viene linearizzato in un `LinearModel`, ridotto alla forma standard (`StandardLinearModel`), tradotto nel *tableau* del semplice e infine risolto. La Figura 4.1 illustra questa sequenza.

È importante osservare che questa sequenza non è rigida. La coda della pipeline è *configurabile*: a partire dal `LinearModel` è possibile, in alternativa al percorso verso il *tableau*, invocare direttamente uno dei solver, ottenendo immediatamente una soluzione senza passare per la forma standard. La scelta delle fasi da eseguire

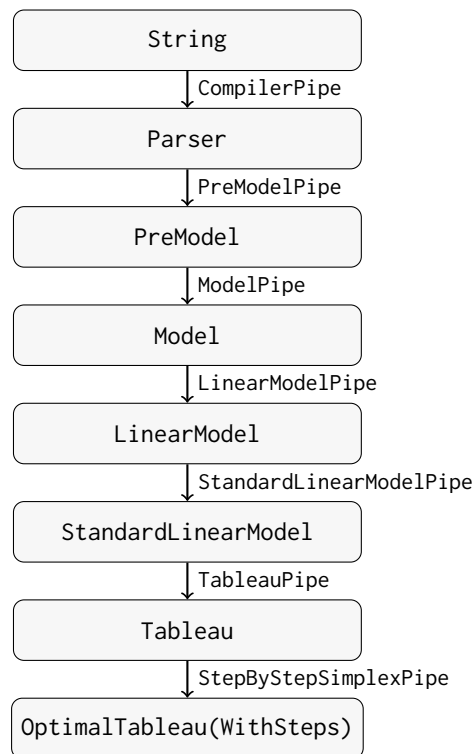


Figure 4.1. La pipeline di compilazione completa, dalla sorgente alla soluzione ottenuta tramite il simplesso passo-passo. Ogni nodo è una rappresentazione intermedia; ogni arco è la *pipe* che effettua la trasformazione.

dipende quindi da ciò che si vuole ottenere, come si vedrà nella prossima sezione.

4.2 L'astrazione *Pipe*

L'intera pipeline è costruita su una singola, semplice astrazione. Ogni trasformazione è una *pipe*, ossia un'unità che riceve un dato, lo elabora e ne restituisce un altro. Formalmente è un tipo che implementa il *trait* `Pipeable`, costituito da un unico metodo:

```

trait Pipeable {
  fn pipe(&self, data: &mut PipeableData, context: &
    PipeContext)
    → Result<PipeableData, PipeError>;
}

```

Listing 4.1. Il *trait* `Pipeable`.

I dati e i loro tipi. Tutti i dati che fluiscono tra le pipe sono rappresentati da un unico tipo enumerativo, `PipeableData`, le cui varianti corrispondono alle rappresentazioni intermedie della Figura 4.1 (`String`, `Parser`, `PreModel`, `Model`, `LinearModel`, `StandardLinearModel`, `Tableau` e le varie soluzioni). A ciascuna variante è associato un identificatore di tipo (`PipeDataType`), che permette di sapere quale dato una pipe abbia ricevuto o prodotto senza ispezionarne il contenuto.

Il contesto di esecuzione. Ogni pipe riceve, oltre al dato, un `PipeContext` condiviso, che mette a disposizione le costanti e le funzioni definite dall'utente. In questo modo le informazioni necessarie alla trasformazione (per esempio i dati dichiarati nel blocco `where` o le funzioni registrate dall'utente) sono accessibili in modo uniforme a tutte le fasi che ne hanno bisogno.

Le pipe concrete. La Tabella 4.1 elenca le pipe disponibili con i rispettivi tipi di ingresso e di uscita. Si noti come le prime sei realizzino il percorso “canonico” della Figura 4.1, mentre le ultime cinque siano *terminali* alternative che, a partire da un `LinearModel`, producono direttamente una soluzione delegando a un solver (Capitolo 8).

Esecuzione di una sequenza di pipe. Una sequenza di pipe viene eseguita da un `PipeRunner`, che ne contiene un elenco e le applica in ordine. Il dato iniziale viene passato alla prima pipe, il cui risultato è passato alla seconda, e così via. Un aspetto cruciale è che il `PipeRunner` non restituisce soltanto il risultato finale, ma l'intera *successione* dei risultati intermedi: ogni rappresentazione prodotta lungo il percorso viene conservata.

4.3 Estensibilità e visualizzazione passo-passo

L'astrazione descritta ha due conseguenze rilevanti, che motivano la scelta architetturale.

Estensibilità. Poiché ogni fase è semplicemente un tipo che implementa `Pipeable`, aggiungere una nuova trasformazione significa definire una nuova pipe, senza modi-

Pipe	Ingresso	Uscita
CompilerPipe	String	Parser
PreModelPipe	Parser	PreModel
ModelPipe	PreModel	Model
LinearModelPipe	Model	LinearModel
StandardLinearModelPipe	LinearModel	StandardLinearModel
TableauPipe	StandardLinearModel	Tableau
StepByStepSimplexPipe	Tableau	OptimalTableauWithSteps
RealSolver	LinearModel	RealSolution
BinarySolverPipe	LinearModel	BinarySolution
IntegerBinarySolverPipe	LinearModel	IntegerBinarySolution
MILPSolverPipe	LinearModel	MILPSolution
AutoSolverPipe	LinearModel	MILPSolution

Table 4.1. Le pipe disponibili e i tipi di dato che trasformano.

ficare quelle esistenti. Le pipe sono manipolate come oggetti-trait intercambiabili, e questo permette di comporle in modo flessibile: si possono inserire nuove fasi intermedie, sostituire un solver con un altro o prevedere percorsi alternativi. Anche la possibilità di terminare la pipeline con solver diversi (reale, binario, intero-binario, MILP o automatico) è una diretta conseguenza di questa impostazione.

Visualizzazione passo-passo. Il fatto che il `PipeRunner` conservi tutte le rappresentazioni intermedie è ciò che rende possibile il *taglio didattico* della piattaforma (Capitolo 9). Aniché mostrare all'utente solo la soluzione finale, è possibile esporre ogni passaggio della compilazione: il modello dopo l'analisi semantica, la sua versione linearizzata, la forma standard e così via. In particolare, la pipe `StepByStepSimplexPipe` produce un risultato che registra ogni iterazione del semplice, consentendo di visualizzare l'evoluzione del *tableau* passo dopo passo, come si vedrà nel Capitolo 8.

Chapter 5

Analisi sintattica e rappresentazione intermedia

Questo capitolo apre la parte dedicata al *front-end* del compilatore. Come anticipato nel Capitolo 2, esso comprende le prime fasi della traduzione: il riconoscimento della struttura del testo sorgente e la sua codifica in una rappresentazione interna su cui le fasi successive potranno lavorare. Coerentemente con l'impostazione adottata, si introducono prima i concetti teorici e poi la loro applicazione concreta in ROOC.

5.1 Concetti teorici

Grammatiche e analisi sintattica. La sintassi di un linguaggio è descritta da una *grammatica*: un insieme di regole che stabiliscono quali sequenze di simboli costituiscono frasi valide. L'*analisi sintattica* (*parsing*) verifica che il testo sorgente rispetti tali regole e ne costruisce una rappresentazione strutturata. La formalizzazione più diffusa è quella delle grammatiche *context-free* (CFG), spesso scritte in notazione BNF, in cui ogni regola può prevedere più alternative. Le CFG possono però essere *ambigue*, cioè ammettere più alberi di derivazione per la stessa frase, il che complica la costruzione del parser.

Parsing Expression Grammar. Le *Parsing Expression Grammar* (PEG) [5] offrono una formalizzazione alternativa, pensata direttamente in funzione del ri-

conoscimento. La differenza fondamentale rispetto alle CFG è la *scelta ordinata*: quando una regola prevede più alternative, queste vengono provate nell'ordine in cui sono scritte e viene accettata la prima che ha successo. Questo rende una PEG intrinsecamente non ambigua. Inoltre una PEG integra l'analisi lessicale e quella sintattica in un'unica descrizione (*scannerless*), eliminando la necessità di un analizzatore lessicale separato: la gestione degli spazi e dei commenti è parte della grammatica stessa.

La rappresentazione intermedia. Il risultato del parsing non è il testo grezzo, ma una sua rappresentazione ad albero che ne cattura la struttura logica trascurando i dettagli sintattici superflui. Questa *rappresentazione intermedia* (IR) è la struttura dati su cui operano le fasi successive del compilatore. Una buona IR rispecchia da vicino i costrutti del linguaggio, così da rendere naturali le elaborazioni successive, e conserva le informazioni necessarie alla diagnostica, in particolare la posizione di ciascun elemento nel sorgente.

La scelta di una PEG per ROOC è motivata proprio da queste proprietà: la non ambiguità semplifica la definizione della grammatica, e l'approccio *scannerless* permette di descrivere in un solo file l'intera sintassi del linguaggio, comprese le sue particolarità come la moltiplicazione implicita.

5.2 Applicazione in ROOC: la grammatica PEST

ROOC definisce la propria sintassi tramite **PEST**, una libreria Rust che implementa le PEG. La grammatica è scritta in un file dedicato e viene tradotta in un parser a tempo di compilazione.

La regola principale. La regola di partenza, `problem`, descrive la struttura complessiva di un modello: la funzione obiettivo, la sezione dei vincoli e i blocchi opzionali `where` e `define` (Listato 5.1).

```
problem = {  
    SOI ~ nl* ~  
    #objective = objective ~ nl+ ~
```

```

    (^"s.t." | ^"subject to") ~ nl+ ~
    #constraints = constraint_list ~
    (nl+ ~ ^"where" ~ #where = consts_declaration)? ~
    (nl+ ~ ^"define" ~ #define = domains_declaration)? ~
    nl* ~ EOI
}

```

Listing 5.1. La regola principale della grammatica.

Questo frammento mostra diverse caratteristiche delle PEG e di PEST. L'operatore `~` indica la sequenza e il suffisso `?` l'opzionalità; `SOI` ed `EOI` marcano l'inizio e la fine dell'input. Le scritture come `#objective = . . .` sono *tag*: una funzionalità di PEST che assegna un nome a una sottoparte della regola, rendendola facilmente recuperabile in fase di costruzione dell'IR senza dover contare la posizione dei nodi.

Regole atomiche, silenziose e scelta ordinata. Altre regole illustrano ulteriori meccanismi. La scelta ordinata si esprime con l'operatore `|`; le regole *atomiche*, marcate con `@`, sono trattate come un unico token senza gestione automatica degli spazi interni; le regole *silenziose*, marcate con `_`, non compaiono nell'albero risultante perché servono solo a strutturare la grammatica.

```

objective_type = @{ ^"min" | ^"max" }
exp = _{ unary_op? ~ exp_leaf ~ (binary_op ~ unary_op? ~
    exp_leaf)* }

```

Listing 5.2. Esempi di regola atomica, silenziosa e di scelta ordinata.

Infine, l'approccio *scannerless* è evidente nel fatto che la gestione degli spazi e dei commenti è definita nella grammatica stessa, tramite le regole speciali `WHITESPACE` e `COMMENT`, anziché in un analizzatore lessicale separato.

Dall'albero di parsing all'IR. L'albero prodotto da PEST non è ancora la rappresentazione intermedia del modello, ma una descrizione fedele della struttura sintattica. Un insieme di funzioni di traduzione lo visita, recuperando le sottoparti tramite i tag (per esempio l'obiettivo, l'elenco dei vincoli, le costanti e i domini) e costruendo le strutture dati dell'IR descritte nella prossima sezione.

5.3 Il PreModel e l'Intermediate Language

La rappresentazione intermedia di ROOC prende il nome di PreModel: è il modello così come risulta dalla sola analisi sintattica, prima dell'analisi semantica e della trasformazione. Esso raccoglie le quattro componenti di un modello, più un riferimento al codice sorgente originale, utile per la diagnostica:

```
struct PreModel {  
    source: Option<String>,  
    objective: PreObjective,  
    constraints: Vec<PreConstraint>,  
    constants: Vec<Constant>,  
    domains: Vec<VariablesDomainDeclaration>,  
}
```

Listing 5.3. La struttura del PreModel (campi principali).

La funzione obiettivo è rappresentata da un `PreObjective`, che ne indica il tipo (minimizzazione o massimizzazione) e l'espressione associata. Ogni vincolo è un `PreConstraint`, che contiene le due espressioni messe a confronto, il tipo di relazione, l'eventuale nome e le eventuali clausole di iterazione (`for`) che lo replicano. Sia l'obiettivo sia i vincoli contengono dunque delle *espressioni*, che costituiscono il cuore dell'IR.

Le espressioni: PreExp. Le espressioni sono rappresentate dal tipo enumerativo `PreExp`, le cui varianti rispecchiano da vicino i costrutti del linguaggio descritti nel Capitolo 3: valori primitivi, variabili semplici e composte, accessi ad array, chiamate di funzione, *block function* (con e senza *scope*) e operazioni binarie e unarie (Listato 5.4).

```
enum PreExp {  
    Primitive(Spanned<Primitive>),  
    Variable(Spanned<String>),  
    CompoundVariable(Spanned<CompoundVariable>),  
    ArrayAccess(Spanned<AddressableAccess>),  
    FunctionCall(InputSpan, FunctionCall),
```

```
BlockFunction(Spanned<BlockFunction>),  
BlockScopedFunction(Spanned<BlockScopedFunction>),  
BinaryOperation(Spanned<BinOp>, Box<PreExp>, Box<PreExp>),  
UnaryOperation(Spanned<UnOp>, Box<PreExp>),  
// ...  
}
```

Listing 5.4. Versione semplificata dell'enumerazione PreExp.

Essendo un albero, una PreExp può contenere altre PreExp: per esempio un'operazione binaria racchiude i due sotto-espressioni operandi. Le variabili composte, come `x_i`, sono rappresentate da un `CompoundVariable` che ne memorizza il nome e l'elenco delle espressioni usate come indice; analogamente, l'accesso a un array è descritto da un `AddressableAccess`. Si noti che a questo livello gli indici sono ancora espressioni non valutate: la loro risoluzione, che trasforma `x_i` nelle concrete variabili del modello, avverrà solo nella fase di trasformazione (Capitolo 6).

5.4 Gestione e tracciamento degli errori

Un aspetto trasversale a tutte le fasi del compilatore è la capacità di segnalare gli errori indicando con precisione il punto del sorgente a cui si riferiscono.

A questo scopo, ogni elemento della rappresentazione intermedia è annotato con la propria posizione nel sorgente tramite un contenitore generico, `Spanned`, che incapsula un valore insieme alle informazioni sulla sua posizione (riga, colonna e lunghezza del frammento di testo). Come si è visto, gran parte delle varianti di PreExp avvolge i propri dati in uno `Spanned`, così che ogni nodo dell'albero conservi il punto del sorgente da cui proviene.

Grazie a questa annotazione, gli errori sono *posizionali*: ogni segnalazione è collegata al frammento di sorgente che l'ha generata, e può così essere presentata indicando dove l'errore ha origine. La stessa infrastruttura permette di tracciare non solo gli errori di parsing, ma anche quelli delle fasi successive, come gli errori di tipo e di trasformazione descritti nel Capitolo 6.

Chapter 6

Analisi semantica: tipi e trasformazione del modello

Capitolo sull'analisi semantica. Anche qui: prima i concetti teorici, poi l'applicazione in ROOC.

6.1 Concetti teorici

Si presentano i concetti di analisi semantica nei compilatori: il *sistema dei tipi*, la verifica statica dei tipi (*type checking*), la nozione di *scope* e di tabella dei simboli, e la distinzione tra rappresentazione di un valore a tempo di esecuzione e la sua descrizione a livello di tipo.

TEORIA: cosa sono i sistemi di tipi, type checking statico vs dinamico, scope e ambienti (frame), inferenza/verifica delle firme di funzione.

6.2 Applicazione in ROOC: il sistema dei tipi

Il sistema dei tipi di ROOC è rappresentato da `PrimitiveKind`, che descrive a livello statico le primitive del linguaggio (inclusi tipi composti come iterabili e tuple) e governa le regole di applicabilità di operatori e funzioni.

APPLICAZIONE: descrivere `PrimitiveKind`, le regole degli operatori (`can_apply_binary_op`) e le firme delle funzioni. Riferimento: `src/primitives/primitive.rs`.

6.3 Il type checker

APPLICAZIONE: descrivere `TypeCheckerContext`, l'uso dei `Frame` per gli scope annidati (uno per livello di `for`), la mappa dei token tipati e la diagnostica dei tipi. Riferimento: `src/type_checker/`.

6.4 Trasformazione `PreModel` $\rightarrow$ `Model`

Superata la verifica dei tipi, il `PreModel` viene trasformato in un `Model`: le costanti vengono valutate, le variabili composte e gli iteratori risolti, e le *block functions* espanse in espressioni concrete.

APPLICAZIONE: descrivere `transform_parsed_problem`, il `TransformerContext`, il `recursive_set_resolver` per la risoluzione di iteratori e variabili composte, e l'espansione delle *block functions*. Riferimento: `src/parser/model_transformer/`.

Chapter 7

Linearizzazione e forma standard

Back-end del compilatore: traduzione del modello tipato in una forma lineare canonica adatta ai solver. Anche qui: prima la teoria, poi l'applicazione.

7.1 Concetti teorici

Si richiamano i concetti di linearità di un'espressione e le riformulazioni standard che consentono di esprimere costrutti non lineari come $\min$, $\max$ e valore assoluto tramite variabili ausiliarie e vincoli aggiuntivi. Si introduce inoltre la *forma standard* di un problema di programmazione lineare, prerequisito per il metodo del simplesso.

TEORIA: definire la linearità, le tecniche di linearizzazione di $\min/\max/\text{abs}$ con variabili ausiliarie, e la forma standard (vincoli di uguaglianza, variabili non negative, obiettivo di minimizzazione).

7.2 Applicazione in ROOC: da Model a LinearModel

Il `Model` può contenere costrutti non lineari; il *linearizer* attraversa l'albero delle espressioni, verifica la linearità e produce un `LinearModel` garantito lineare.

APPLICAZIONE: descrivere il `Linearizer`, il rilevamento della non-linearità (es. prodotto di variabili) e la struttura del `LinearModel`. Riferimento: `src/transformers/linearizer.rs`.

7.3 Linearizzazione di min, max e valore assoluto

APPLICAZIONE: mostrare con esempi come min/max vengano sostituiti da una variabile ausiliaria e dai relativi vincoli. Discutere lo stato della linearizzazione del valore assoluto (limite noto, cfr. conclusioni).

7.4 Trasformazione in forma standard

Il `LinearModel` viene infine normalizzato in forma standard: si introducono variabili di *slack* e *surplus*, si effettua lo *splitting* delle variabili libere e si normalizza la funzione obiettivo.

APPLICAZIONE: descrivere `to_standard_form`, l'aggiunta di slack/surplus, lo splitting delle variabili libere ($x = x' - x''$) e la normalizzazione dell'obiettivo. Riferimento: `src/transformers/standardizer.rs`.

Chapter 8

Risoluzione dei modelli

Capitolo focalizzato ma compatto sui solver. In primo piano il contributo originale microlp; i backend esterni sono trattati come opzioni intercambiabili. Mantenere la struttura teoria → applicazione.

8.1 Concetti teorici

Si richiamano gli algoritmi alla base della risoluzione: il metodo del simplesso e la sua variante a due fasi con variabili artificiali per la ricerca di una base ammissibile, e il *branch-and-bound* per l'estensione ai problemi con variabili intere e binarie (MILP).

TEORIA: simplesso (tableau, pivot, test del rapporto minimo), metodo a due fasi, branch-and-bound per MILP. Mantenere il livello adeguato a una tesi triennale.

8.2 Applicazione in ROOC: il simplesso a due fasi

ROOC include un'implementazione propria del simplesso a due fasi, capace di esporre i *tableau* intermedi per consentire la visualizzazione passo-passo.

APPLICAZIONE: descrivere l'implementazione del Tableau, le due fasi, l'estrazione della soluzione (`OptimalTableau`) e la registrazione degli step. Riferimento: `src/solvers/simplex/`.

8.3 **microlp**: fork ed estensione a MILP

Il solver **microlp** è un *fork* di un crate archiviato, originariamente limitato al solo semplice (variabili continue). Nell'ambito di questo lavoro è stato ripreso e mantenuto, ed esteso al supporto di variabili *interi* e *binarie*, diventando di fatto un solver MILP.

APPLICAZIONE/CONTRIBUTO: spiegare lo stato di partenza del fork (simplex continuo), le modifiche apportate per il supporto a variabili intere/binarie (branch-and-bound) e l'integrazione in ROOC. Questo è uno dei contributi originali della tesi.

8.4 Backend esterni e *auto-solver*

Oltre al solver proprio, ROOC può delegare la risoluzione a backend esterni in base al dominio delle variabili, tramite un meccanismo di *auto-solver*.

APPLICAZIONE: descrivere i backend (*good_lp*/Clarabel, *copper* per problemi binari, HiGHS nel browser) e la logica di *auto_solver* che seleziona il solver adatto. Riferimenti: `src/solvers/`.

8.5 Confronto qualitativo

Confronto qualitativo tra i solver disponibili (espressività, prestazioni, uso didattico). Eventuali piccoli esempi/benchmark.

Chapter 9

Dalla libreria alla piattaforma web

Capitolo di supporto: mostra come il motore (libreria Rust) diventi una libreria riusabile e una piattaforma web didattica. Mantenere il taglio sintetico, coerente con il focus sul motore.

9.1 Compilazione in WebAssembly e la libreria TypeScript

La libreria Rust viene compilata in WebAssembly tramite `wasm-pack` ed esposta a JavaScript/TypeScript attraverso `serde-wasm-bindgen`. Sopra di essa è costruita la libreria `@specy/rooc`, che offre un'API type-safe.

Descrivere il processo di build (Rust $\rightarrow$ WASM $\rightarrow$ pacchetto npm), la marshalling dei dati e l'API pubblica di `ts-lib`. Riferimenti: `src/lib.rs`, `src/pipe/pipe_wasm_runner.rs`, `ts-lib/`.

9.2 L'IDE web e l'LSP nel browser

La piattaforma web è un'applicazione SvelteKit con editor Monaco. Un Language Server Protocol (LSP) parziale, eseguito interamente nel browser, fornisce evidenziazione della sintassi, hover dei tipi, completamento ed evidenziazione degli

errori.

Descrivere l'IDE: editor Monaco con linguaggio ROOC custom, l'LSP nel browser (diagnostica, hover, completamento, formattazione), il rendering LaTeX con KaTeX e la persistenza locale (IndexedDB). Riferimenti: `frontend/src/lib/Monaco.ts`, `frontend/src/lib/Rooc/`.

9.3 Valore didattico

Argomentare il valore didattico: visualizzazione passo-passo della risoluzione, esecuzione di funzioni utente in sandbox, documentazione interattiva e condivisione dei modelli. Collegare all'astrazione a pipe (Cap. 4).

Chapter 10

Conclusioni e sviluppi futuri

10.1 Riepilogo dei contributi

Riepilogare i contributi della tesi: il linguaggio ROOC, il compilatore multi-fase, il solver MILP microlp e la piattaforma web didattica.

10.2 Limitazioni

Discutere le limitazioni note: linearizzazione del valore assoluto, assenza di vincoli logici, prestazioni del solver proprio rispetto ai backend industriali.

10.3 Sviluppi futuri

Delineare i possibili sviluppi: vincoli logici, ulteriori solver, miglioramenti del linguaggio e della piattaforma.

Appendix A

Grammatica completa di ROOC

Riportare la grammatica PEST completa (`src/parser/grammar.pest`), eventualmente commentata.

Appendix B

Esempi di modelli

Raccogliere esempi di modelli ROOC completi (knapsack, dominating set, ecc.) con relativo modello lineare compilato e soluzione.

Bibliography

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2 edition, 2006.
- [2] Dimitris Bertsimas and John N. Tsitsiklis. *Introduction to Linear Optimization*. Athena Scientific, Belmont, MA, 1997.
- [3] George B. Dantzig. *Linear Programming and Extensions*. Princeton University Press, Princeton, NJ, 1963.
- [4] Iain Dunning, Joey Huchette, and Miles Lubin. JuMP: A modeling language for mathematical optimization. *SIAM Review*, 59(2):295–320, 2017.
- [5] Bryan Ford. Parsing expression grammars: A recognition-based syntactic foundation. In *Proceedings of the 31st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '04)*, pages 111–122. ACM, 2004.
- [6] Robert Fourer, David M. Gay, and Brian W. Kernighan. *AMPL: A Modeling Language for Mathematical Programming*. Thomson/Brooks/Cole, 2 edition, 2003.
- [7] Gurobi Optimization, LLC. Gurobi optimizer reference manual. <https://www.gurobi.com>, 2024.
- [8] William E. Hart, Carl D. Laird, Jean-Paul Watson, David L. Woodruff, Gabriel A. Hackebeil, Bethany L. Nicholson, and John D. Sirola. *Pyomo — Optimization Modeling in Python*. Springer, 2 edition, 2017.
- [9] Qi Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.

- [10] Andrew Makhorin. *GNU Linear Programming Kit, Reference Manual*. Free Software Foundation, 2017.
- [11] George L. Nemhauser and Laurence A. Wolsey. *Integer and Combinatorial Optimization*. Wiley, New York, 1988.
- [12] Nicholas Nethercote, Peter J. Stuckey, Ralph Becket, Sebastian Brand, Gregory J. Duck, and Guido Tack. MiniZinc: Towards a standard CP modelling language. In *Principles and Practice of Constraint Programming (CP 2007)*, volume 4741 of *LNCS*, pages 529–543. Springer, 2007.
- [13] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, Cambridge, MA, 2002.
- [14] WebAssembly Community Group. Webassembly specification. <https://webassembly.org>, 2024.
- [15] Laurence A. Wolsey. *Integer Programming*. Wiley-Interscience, New York, 1998.